

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE
AREQUIPA
ESCUELA PROFESIONAL DE CIENCIA DE LA
COMPUTACIÓN
CURSO: BIGDATA 2026A

Trabajo Unidad III

Plataforma Inteligente para Simulación y Análisis de
Audiencias Digitales en Tiempo Real utilizando
Arquitecturas Orientadas a Eventos

Rivas Huanca Diego Raul
Sumare Uscca Josue Gabriel
Humpiri Valdivia Freddy Leonel
Escuela Profesional de Ciencia de la Computación

Julio 2026

Índice

1	Introducción	2
2	Marco conceptual	2
2.1	Arquitectura Orientada a Eventos (EDA)	2
2.2	Event Streaming y Apache Kafka	2
2.3	Apache Flink	2
2.4	Audiencias digitales	2
3	Arquitectura del sistema	2
3.1	Infraestructura desplegada	3
4	Diseño e implementación	3
4.1	Simulador de agentes	3
4.2	Apache Kafka	4
4.3	Apache Flink: métricas y clasificación de audiencias	4
4.4	Dashboard en tiempo real	5
5	Resultados: comparativa de escenarios	5
5.1	Análisis	5
5.2	Audiencias detectadas por escenario	5
6	Capturas de pantalla	6
6.1	Dashboard — Escenario Normal	6
6.2	Dashboard — Escenario Campaña Escolar	6
6.3	Dashboard — Escenario Cyber Monday	6
7	Conclusiones	7
8	Referencias	7

1 Introducción

Las organizaciones modernas generan volúmenes masivos de eventos digitales, inicios de sesión, búsquedas, visualizaciones de producto, compras que tradicionalmente se analizaban mediante procesamiento por lotes, con retrasos de horas o días. Este trabajo implementa una plataforma de **procesamiento de datos en tiempo real** basada en una **Arquitectura Orientada a Eventos (EDA)**, capaz de simular el comportamiento de decenas de usuarios digitales mediante agentes autónomos, capturar sus eventos con Apache Kafka, procesarlos continuamente con Apache Flink para construir audiencias digitales, y visualizar los resultados en un dashboard interactivo en tiempo real.

El objetivo del proyecto es demostrar el flujo completo *agentes* $\rightarrow$ *Kafka* $\rightarrow$ *Flink* $\rightarrow$ *audiencias* $\rightarrow$ *dashboard*, y analizar cómo cambia el comportamiento agregado de la plataforma bajo distintos escenarios comerciales (día normal, Cyber Monday, campaña escolar, etc.).

2 Marco conceptual

2.1 Arquitectura Orientada a Eventos (EDA)

Un modelo de diseño donde los componentes de software no se comunican directamente entre sí, sino que publican **eventos** acciones significativas ocurridas en el sistema en un canal común, permitiendo que otros componentes los consuman de forma independiente y desacoplada.

2.2 Event Streaming y Apache Kafka

El *Event Streaming* es el procesamiento continuo de eventos conforme se producen, a diferencia del procesamiento por lotes. Apache Kafka es la plataforma de streaming utilizada como intermediario: organiza los eventos en **tópicos**, particionados para permitir paralelismo y alta disponibilidad.

2.3 Apache Flink

Motor de procesamiento de flujos de datos (*Stream Processing Engine*) usado para consumir los eventos desde Kafka y aplicar transformaciones continuas: filtrado, enriquecimiento, ventanas de tiempo, agregaciones y clasificación basada en reglas de negocio.

2.4 Audiencias digitales

Conjuntos de usuarios agrupados según patrones de comportamiento observados en tiempo real (y no solo datos demográficos estáticos), útiles para personalizar campañas, detectar riesgo de abandono o identificar clientes de alto valor.

3 Arquitectura del sistema

El sistema implementado sigue el flujo:

```
Agentes simulados (Python, 8 perfiles)
  | publican eventos JSON
  v
Apache Kafka (audiencias-user-events, audiencias-purchase-events)
  | consumido continuamente
  v
Apache Flink (metricas por ventana + clasificador de audiencias)
  | publica resultados JSON
```

```

      v
Kafka (audiencias-metricas-output, audiencias-resultado-output)
      |   consumido por
      v
Backend FastAPI (WebSocket) ---> Dashboard web en tiempo real

```

A diferencia de una arquitectura con base de datos intermedia, aquí Flink publica sus resultados directamente a nuevos tópicos de Kafka, y un backend ligero en FastAPI los retransmite por WebSocket al navegador. Esto evita la complejidad de una capa de persistencia adicional, manteniendo el principio de streaming de punta a punta: cada resultado se empuja al cliente apenas se calcula, sin necesidad de que el dashboard haga polling a una base de datos.

3.1 Infraestructura desplegada

Todo el sistema corre sobre una instancia EC2 (Ubuntu 24.04, **t3.large**, 2 vCPU / 8 GiB RAM) en AWS Academy, con cuatro procesos concurrentes: el broker de Kafka, el job de Flink, el backend del dashboard, y el simulador de agentes.

4 Diseño e implementación

4.1 Simulador de agentes

Se implementaron 8 perfiles de comportamiento, cada uno corriendo en su propio hilo y generando sesiones completas de navegación (login → ver/buscar productos → carrito → compra o abandono → logout):

Perfil		Comportamiento
Comprador	com-	Compra rápido, consulta pocos productos, alta probabilidad de compra
Comparador		
Comprador	noc-	Solo genera actividad entre las 22:00 y las 05:00
Cliente Premium		Compra pocas veces, pero de alto valor (S/1500 – S/6000)
Cliente Frecuente		Realiza compras constantemente, alta frecuencia de sesión
Usuario	Explo-	Navega mucho (8–20 productos), nunca compra
Comprador		
Cliente Indeciso		Agrega y quita productos del carrito repetidamente
Cliente Estacional		Su comportamiento se amplifica según el escenario comercial activo

Cada agente publica eventos JSON con la estructura:

```

{
  "timestamp": "2026-07-20T02:21:53",
  "user_id": "USR55301",
  "company": "Retail",
  "event": "VIEW_PRODUCT",
  "product": "Laptop Lenovo",
  "category": "Electronics",
  "city": "Arequipa",
  "price": 3341.50,

```

```
"agent_id": "CLIENTE_ESTACIONAL_01",
"agent_type": "CLIENTE_ESTACIONAL"
}
```

Adicionalmente, se implementaron **7 escenarios comerciales** (`normal`, `navidad`, `cyber_monday`, `black_friday`, `dia_del_padre`, `fiestas_patrias`, `campana_escolar`), cada uno aplicando multiplicadores globales sobre la probabilidad de compra, la frecuencia de sesiones y el valor promedio de compra de *todos* los agentes, simulando el efecto de una campaña real sobre el tráfico de la plataforma.

4.2 Apache Kafka

Se configuraron 4 tópicos:

Tópico	Contenido
<code>audiencias-user-events</code>	LOGIN, SEARCH, VIEW_PRODUCT, ADD_CART, REMOVE_CART,
<code>audiencias-purchase-events</code>	PURCHASE, PAYMENT_FAILED
<code>audiencias-metricas-output</code>	Métricas agregadas por ventana (salida de Flink)
<code>audiencias-resultado-output</code>	Audiencias detectadas por usuario (salida de Flink)

4.3 Apache Flink: métricas y clasificación de audiencias

Se implementó un único job (`MainDashboardPipeline`) con dos ramas de procesamiento sobre el mismo stream de eventos:

Rama de métricas (ventanas de 15 segundos, `TumblingProcessingTimeWindows`): eventos totales, eventos por segundo, usuarios activos distintos, eventos por tipo, top 5 productos más vistos/comprados, compras por ciudad, y tasa de conversión (`PURCHASE` / `ADD_CART`).

Rama de audiencias (estado por usuario mediante `KeyedProcessFunction`, `keyBy(user_id)`): mantiene contadores de sesión (vistas, vistas en categoría Electronics, productos agregados al carrito, carrito activo, valor máximo de compra) y un contador histórico de compras que persiste entre sesiones. Al detectar el evento `LOGOUT`, evalúa las siguientes reglas:

Audiencia	Regla
Alta intención de compra / Abandono de carrito	Agregó productos al carrito pero no completó ninguna compra en la sesión
Cliente Frecuente	3 o más compras completadas (histórico acumulado)
Cliente Premium	Alguna compra de la sesión superó S/1500
Interesado en Tecnología	Al menos 60% de sus vistas fueron de la categoría <i>Electronics</i>
Riesgo de Abandono	Navegó bastante (5+ vistas) pero nunca agregó nada al carrito

Un mismo usuario puede activar varias audiencias simultáneamente (por ejemplo, `ALTA_INTENCION_COMPRA` | `INTERESADO_TECNOLOGIA`), lo cual se observó consistentemente en las pruebas.

4.4 Dashboard en tiempo real

Backend en FastAPI que consume los tópicos de salida de Flink (`KafkaConsumer` en un hilo aparte) y retransmite cada mensaje nuevo a todos los clientes conectados vía WebSocket, sin persistencia intermedia. El frontend (HTML/CSS/JS + Chart.js) muestra:

- KPIs en vivo: eventos totales, eventos/segundo, usuarios activos, conversión
- Gráfico de eventos por tipo y de compras por ciudad
- Tablas de productos más vistos y más comprados
- Feed en vivo de audiencias detectadas por usuario
- Alertas automáticas (ej. conversión baja, alta tasa de pagos rechazados)

5 Resultados: comparativa de escenarios

Se ejecutó el simulador (20 agentes, 8 perfiles) bajo tres escenarios de forma secuencial y aislada (un único proceso productor activo por corrida, confirmado mediante el conteo de usuarios activos en el dashboard, siempre ≤ 20), dejando correr cada uno un par de minutos antes de capturar resultados.

Escenario	Eventos	Ev./seg	Usuarios	Conversión	Mult. prob. compra
Campaña Escolar	128	8.53	18	23.1%	1.3×
Normal (base)	111	7.40	18	31.6%	1.0×
Cyber Monday	150	10.00	18	33.3%	2.2×

5.1 Análisis

Los resultados muestran una tendencia **coherente con la configuración de multiplicadores** definida para cada escenario: Campaña Escolar (multiplicador de compra más bajo, 1.3×) produjo la menor conversión observada (23.1%), mientras que Cyber Monday (multiplicador más alto, 2.2×) produjo la mayor (33.3%), con el escenario Normal como punto intermedio. El volumen de eventos por segundo también creció en el mismo orden (Campaña Escolar < Normal < Cyber Monday), reflejando el multiplicador de frecuencia de sesiones de cada escenario.

Es importante notar que la conversión no escaló de forma estrictamente lineal con el multiplicador (2.2× el multiplicador no produjo 2.2× la conversión base), lo cual es esperado: la probabilidad de compra está acotada a un máximo de 0.95 en el motor del agente (`perfiles.py`), y no todos los perfiles reaccionan igual al escenario (por ejemplo, el Usuario Explorador nunca compra independientemente del escenario activo).

En una corrida adicional de mayor duración bajo `cyber_monday` (8632 eventos acumulados, 20 usuarios activos), la conversión general se estabilizó en 36.9%, con *Mouse Logitech*, *Mochila Escolar* y *Laptop Lenovo* como los productos más comprados, y una distribución de compras razonablemente uniforme entre las 5 ciudades simuladas (Piura, Lima, Arequipa, Cusco, Trujillo), lo que confirma que el simulador no introduce sesgos geográficos artificiales.

5.2 Audiencias detectadas por escenario

En los tres escenarios se observaron consistentemente las 5 audiencias definidas, con las combinaciones `ALTA_INTENCION_COMPRA` / `ABANDONO_CARRITO` e `INTERESADO_TECNOLOGIA` como las más frecuentes esperable dado que el catálogo de productos tiene una alta proporción de artículos de la categoría *Electronics*. Se detectaron además usuarios con múltiples audiencias simultáneas (por ejemplo, `CLIENTE_PREMIUM` | `INTERESADO_TECNOLOGIA`), demostrando que el clasificador no trata las audiencias como mutuamente excluyentes, reflejando mejor el comportamiento real de un usuario.

6 Capturas de pantalla

6.1 Dashboard — Escenario Normal



6.2 Dashboard — Escenario Campaña Escolar



6.3 Dashboard — Escenario Cyber Monday



7 Conclusiones

1. Se implementó exitosamente el flujo completo de una arquitectura orientada a eventos, desde la generación simulada de tráfico hasta la visualización en tiempo real, sin componentes de procesamiento por lotes en ninguna etapa.
2. El diseño de agentes con perfiles de comportamiento diferenciados, combinado con escenarios que aplican multiplicadores globales, permitió observar cambios de comportamiento agregado consistentes y explicables por el diseño (mayor multiplicador de probabilidad de compra $\rightarrow$ mayor conversión observada), validando que el simulador responde correctamente a los parámetros configurados.
3. El uso de `KeyedProcessFunction` en Flink permitió mantener estado por usuario a lo largo de una sesión (y parcialmente entre sesiones, para el conteo histórico de compras) sin necesidad de una base de datos externa, resolviendo la clasificación de audiencias completamente en memoria dentro del propio job de streaming.
4. Publicar los resultados de Flink directamente a tópicos de Kafka de salida, consumidos por un backend WebSocket sin capa de persistencia, demostró ser una arquitectura suficiente y más simple que introducir una base de datos intermedia, para el caso de uso de visualización en tiempo real de este proyecto.
5. Entre las dificultades técnicas resueltas destacan: conflictos de classloading de Flink al ejecutarse vía plugins de Maven (resuelto empaquetando un *fat jar* con `maven-shade-plugin`), incompatibilidad de Kryo con el sistema de módulos de Java 17 al serializar colecciones en POJOs (resuelto evitando `List` en favor de `String` en los objetos de resultado), y conflictos de versión entre `jackson-core` y `jackson-databind` resueltos fijando versiones explícitas en el `pom.xml`.

8 Referencias

- Apache Software Foundation. (2026). *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>
- Apache Software Foundation. (2026). *Apache Flink Documentation*. <https://nightlies.apache.org/flink/flink-docs-stable/>
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- AWS Documentation. (2026). *Amazon Managed Service for Apache Flink*. <https://docs.aws.amazon.com/managed-flink/>